Proyecto Integrador: Primera Entrega

Derivative-free optimal multiple-root finding family of iterative methods

Métodos Numéricos

Integrantes:

Ailyn Sofía Gómez Rodríguez

Julián Camilo Jiménez

Julián Steven Duarte

Docente:

Edwin Fernando Muñoz Chipatecua

Pregrado en Ciencia de Datos
Departamento de Matemáticas
Universidad Externado de Colombia
Bogotá D.C., Colombia — Septiembre de 2026

1. Comprensión del Artículo Científico

1.1. El problema fundamental: raíces múltiples y degradación de métodos clásicos

En computación científica y Ciencia de Datos, al resolver ecuaciones no lineales $g(x) = 0$ surgen frecuentemente raíces con **multiplicidad** $m \geq 2$, donde la función y sus primeras $m-1$ derivadas se anulan simultáneamente. Geométricamente, la gráfica se aplana en un entorno amplio de la raíz, lo que degrada la convergencia cuadrática del método clásico de Newton-Raphson a puramente lineal con factor asintótico $(m-1)/m$. El método de Newton modificado de Schröder restaura el orden cuadrático, pero acarrea la exigencia ineludible de evaluar la derivada analítica $g'(x_k)$ en cada paso. En Ciencia de Datos, donde las funciones provienen de modelos de caja negra o simuladores complejos, la evaluación de derivadas analíticas es inviable, y reemplazarla por diferencias finitas elementales induce errores de truncamiento y cancelaciones numéricas severas [9].

1.2. La propuesta del artículo: esquema óptimo de orden cuatro sin derivadas

Para superar este dilema, Jerezano, Chicharro y Garrido-Saez (2026) [1] proponen una familia de métodos iterativos en dos pasos que prescinde por completo de derivadas analíticas y alcanza **orden de convergencia cuatro ($p = 4$)**.

El esquema introduce un desplazamiento cuadrático adaptativo $w_k = x_k + \beta[g(x_k)]^2$ ($\beta \in \mathbb{C}\backslash\{0\}$), aproximando la derivada mediante la diferencia dividida secante $g[w_k, x_k] = \frac{g(w_k)-g(x_k)}{w_k-x_k}$. Con dicha pendiente se genera un predictor intermedio y_k, sobre el cual se evalúa la variable de control adimensional $t_k = (g(y_k)/g(x_k))^{1/m}$. Mediante el diseño de una función de peso $H(t)$ que satisface $H(0) = 1$, $H'(0) = 1$, $H''(0) = 4$ y $|H'''(0)| < \infty$, el paso corrector cancela los errores de órdenes inferiores y eleva la convergencia a orden 4.

Optimalidad de Kung-Traub: La conjetura formulada por Kung y Traub (1974) [2] establece que un método iterativo sin memoria que utiliza d evaluaciones funcionales por iteración puede alcanzar como máximo un orden $p_{\text{máx}} = 2^{d-1}$. El esquema de Jerezano et al. evalúa tres funciones por iteración $(g(x_k), g(w_k), g(y_k))$ y ninguna derivada ($d = 3$). Su cota máxima teórica es $2^{3-1} = 4$. Al alcanzar exactamente dicho orden, la familia se clasifica formalmente como **óptima**.

1.3. Aritmética de punto flotante y absorción numérica

Un aspecto numérico crucial radica en la física de punto flotante subyacente a la perturbación secante. En precisión doble estándar IEEE-754 (`float64`, $\varepsilon_M \approx 2.22 \times 10^{-16}$), la distancia absoluta entre w_k y x_k satisface:

$$|w_k - x_k| = |\beta|[g(x_k)]^2 \sim \mathcal{O}(e_k^{2m}). \tag{1}$$

La absorción numérica ocurre cuando la perturbación cae por debajo de la resolución de la mantisa, es decir, cuando $\beta[g(x_k)]^2 < \frac{1}{2}\varepsilon_M|x_k|$. Para la función $g_1(x)$ con $m = 2$, $\beta = 0.01$ y $|x_k| \approx 1.75$, el estancamiento sobreviene cuando:

$$|g_1(x_k)| < \sqrt{\frac{\varepsilon_M|x_k|}{2\beta}} \sim 1.4 \times 10^{-7}. \tag{2}$$

Dado que cerca de la raíz $|g_1(x)| \approx 0.03\, e^2$, el umbral corresponde a $e \approx 2.15 \times 10^{-3}$. Ningún iterado dentro de esa zona puede usarse como punto de partida de un nuevo paso (0/0); la precisión final alcanzable depende del último salto de orden 4 y se medirá empíricamente en el experimento.

Por esta razón, los autores en [1] recurrieron a aritmética de precisión variable en MATLAB con **4000 dígitos de mantisa** para alcanzar la cota $|x_{k+1} - x_k| + |g(x_k)| < 10^{-200}$. En nuestro proyecto en Python, emplearemos la biblioteca `mpmath` [10] con alta precisión para replicar estos resultados y documentar el estancamiento en `float64`.

1.4. Resultados numéricos reportados por los autores

En las Tablas 1, 2 y 3 de [1], los autores compararon las familias propuestas M_1 (acelerador polinomial) y M_2 (acelerador racional) frente a métodos de la literatura: Sharma y Sharma (SS) [4], Behl, Cordero, Motsa y Torregrosa (BH) [5], Kumar et al. (KK) [6] y Behl et al. (BB) [7]:

1. **Orden ACOC:** En todos los casos convergentes se confirmó empíricamente un ACOC de 4.0000 en 5 a 8 iteraciones.

2. **Fallas en la literatura:** El método BB no convergió en ninguno de los tres problemas evaluados (nc). Asimismo, los métodos SS y BH no lograron converger en el problema $g_3(x)$.

3. **Comparación entre M_2 y KK:** El método KK converge en todos los problemas analizados. Para $g_1(x)$ con $x_0 = 1.9$, KK converge en 6 iteraciones mientras que M_2 requiere 7 iteraciones. Aunque el texto de [1] afirma que M_2 es el más rápido en $g_1(x)$, sus datos en la Tabla 1 muestran que KK requirió una iteración menos. El residuo menor de M_2 (1.77×10^{-1796} frente a 1.85×10^{-1661} de KK) probablemente se debe a la iteración adicional efectuada y no a una velocidad intrínseca superior. En $g_3(x)$ con $x_0 = 3.4$, M_1 y M_2 alcanzan la solución en 6 iteraciones frente a 7 de KK.

1.5. Supuestos y delimitación de limitaciones

Diferenciamos las limitaciones explícitas del artículo de nuestras inferencias analíticas:
Limitaciones declaradas por los autores en el artículo [1]:

- **Dominio del análisis de estabilidad:** El estudio de dinámica compleja se restringe al polinomio cuadrático $p(z) = (z-1)^2$. Se demostró que M_2 carece por completo de puntos fijos extraños para dicho polinomio, mientras que M_1 posee puntos fijos extraños pero todos son repulsores (lo que previene converger a raíces falsas). La extensión multidimensional se declaró como trabajo futuro.
- **Comportamiento respecto al parámetro β:** Al crecer $|\beta|$, las cuencas de atracción de la raíz sufren un encogimiento continuo (Figuras 3 a 9 del artículo). Por ello se adopta $\beta = 1/100$ en las pruebas numéricas.

Limitaciones e inferencias identificadas por el equipo de trabajo:

- **Multiplicidad errónea $m' \neq m$:** Introducir una multiplicidad estimada m' distinta de la real provoca que la convergencia se degrade a lineal con un factor que depende de H y de m'/m, que se medirá empíricamente en la experimentación.
- **Hipótesis sobre los fallos de SS y BH en g_3:** El artículo únicamente registra *nc* para SS y BH en $g_3(x)$. Como hipótesis, los fallos podrían deberse a la sensibilidad a la semilla inicial o a la captura por raíces vecinas (las raíces en 4 y 5 están cerca de $x_0 = 3.4$); ambas opciones se contrastarán empíricamente.
- **Rama principal en potencias fraccionarias:** La variable $t_k = (g(y_k)/g(x_k))^{1/m}$ puede arrojar cocientes negativos o complejos. Se adopta la rama principal del logaritmo ($\mathrm{Log}(z) = \ln|z| + i\,\mathrm{Arg}(z)$ con $-\pi < \mathrm{Arg}(z) \leq \pi$) para asegurar una definición determinista de t_k.

2. Formulación del Problema Computacional

2.1. Contexto en Ciencia de Datos y objetivo del proyecto

En Ciencia de Datos, la calibración de modelos y la optimización de pérdidas complejas requieren localizar puntos estacionarios donde el gradiente se anula ($\nabla L(\theta) = 0$). Cuando las funciones de pérdida son evaluadas mediante simulaciones de caja negra y los mínimos presentan curvaturas casi nulas (análogos a raíces múltiples), los solvers estándar se ralentizan o fallan. El objetivo del proyecto consiste en implementar en Python, de forma estructurada y reproducible en el repositorio oficial (`https://github.com/ailynsof/Proyecto-M-todos-Numericos`), las familias óptimas de orden 4 de Jerezano et al. (2026), verificando su convergencia y comparándolas con los esquemas del curso.

2.2. Formulación matemática del algoritmo predictor-corrector

El esquema iterativo en dos pasos se define mediante:

$$w_k = x_k + \beta[g(x_k)]^2, \quad \beta \in \mathbb{C} \setminus \{0\}, \tag{3}$$

$$g[w_k, x_k] = \frac{g(w_k) - g(x_k)}{w_k - x_k}, \tag{4}$$

$$y_k = x_k - m\frac{g(x_k)}{g[w_k, x_k]}, \tag{5}$$

$$t_k = \exp\left(\frac{1}{m}\,\mathrm{Log}\left(\frac{g(y_k)}{g(x_k)}\right)\right), \tag{6}$$

$$x_{k+1} = x_k - m\,H(t_k)\frac{g(x_k)}{g[w_k, x_k]}. \tag{7}$$

Se implementarán dos familias representativas:

1. **Método M_1 (Acelerador Polinomial):** $H_1(t) = 1 + t + 2t^2$.
2. **Método M_2 (Acelerador Racional):** $H_2(t) = \frac{1-t}{1-2t}$.

3. Obtención de la Información y Problemas de Prueba

3.1. Naturaleza determinista de la información requerida

En estricto cumplimiento de la guía, se establece que **no se utilizarán series con ruido estocástico**. El proyecto utilizará **funciones matemáticas analíticas deterministas** de prueba (*benchmark*) procedentes de problemas de termodinámica, física térmica y álgebra lineal documentados en [1]. El conocimiento del valor analítico exacto de la raíz α permite evaluar con certeza el error absoluto $|x_k - \alpha|$, el residuo $|g(x_k)|$ y la tasa empírica ACOC sin perturbaciones ajenas al algoritmo.

3.2. Descripción de las funciones benchmark seleccionadas

Se implementarán los tres problemas de prueba propuestos por los autores:

1. **Problema 1 (Ecuación de Van der Waals en Termodinámica):**

$$g_1(x) = x^3 - 5.22x^2 + 9.0825x - 5.2675 = (x - 1.75)^2(x - 1.72). \tag{8}$$

   Modela el comportamiento de gases reales. Presenta una raíz doble ($m = 2$) en $\alpha = 1.75$ y una raíz simple vecina en 1.72. La cercanía extrema ($\Delta = 0.03$) desafía la estabilidad frente a la atracción hacia la raíz simple.

2. **Problema 2 (Ley de Radiación Térmica de Planck):**

$$g_2(x) = \left(e^{-x} - 1 + \frac{x}{5}\right)^3. \tag{9}$$

   Determina la longitud de onda de máxima densidad espectral en la radiación de cuerpo negro. Posee una raíz triple ($m = 3$) en $\alpha \approx 4.9651142317$. Evalúa la robustez ante funciones trascendentes no polinomiales.

3. **Problema 3 (Polinomio Característico en Álgebra Lineal):**

$$g_3(x) = x^9 - 29x^8 + 349x^7 - 2261x^6 + 8455x^5 - 17663x^4 + 15927x^3 + 6993x^2 - 24732x + 12960. \tag{10}$$

   Ecuación característica de una matriz cuadrada 9×9, con raíz de multiplicidad $m = 4$ en $\alpha = 3$. Se evaluará en su forma polinomial expandida (10) tal como en el artículo base para estudiar los efectos de acumulación de redondeo numérico frente a su representación factorizada $(x - 3)^4(x - 8)(x - 5)(x - 4)(x - 1)(x + 1)$.

El Cuadro 1 resume las características y semillas iniciales de las funciones de prueba. Nótese que para $g_3(x)$ la semilla es $x_0 = 3.4$ en estricta coincidencia con la Tabla 3 de [1]:

Cuadro 1: Características y parámetros de las funciones de prueba del proyecto.

Función de prueba $g(x)$	**Contexto**	**Raíz** α	**Mult.** (m)	**Semillas** (x_0)	**Desafío numérico principal**
$g_1(x) = (x - 1.75)^2(x - 1.72)$	Termodinámica	1.75	2	1.9, 3.0	Proximidad crítica a raíz simple ($\Delta = 0.03$).
$g_2(x) = (e^{-x} - 1 + x/5)^3$	Física Térmica	≈ 4.96511	3	5.3, 10.0	Presencia de términos exponenciales trascendentes.
$g_3(x) =$ Expandido (10)	Álgebra Lineal	3.0	4	2.8, 3.4	Multiplicidad 4 y coeficientes de gran magnitud.

4. Plan de Implementación, Criterios y Avances

4.1. Métodos numéricos a programar en Python

En el módulo `src/` del repositorio del proyecto se desarrollarán los siguientes métodos:

1. **Familias del artículo base:** Métodos M_1 y M_2 con $\beta = 0.01$. Se programará un barrido complementario con $\beta \in \{0.001, 0.01, 1.0\}$ para evaluar empíricamente el radio de convergencia (aclarando que el análisis formal de dinámica compleja en el plano de Julia queda fuera del alcance de la asignatura).

2. **Métodos de contraste del curso y de la literatura:**
   - *Newton modificado de Schröder ($p = 2$):* Método clásico con derivada analítica $g'(x_k)$.
   - *Steffensen modificado ($p = 2$):* Método adaptativo sin derivadas basado en $w_k = x_k + \beta g(x_k)$ [3].
   - *Kumar et al. (KK, $p = 4$) [6]:* El competidor óptimo libre de derivadas de orden 4 más cercano reportado en la literatura, definido formalmente como:

$$y_k = x_k - m\frac{g(x_k)}{g[w_k, x_k]}, \qquad x_{k+1} = y_k - \frac{(m+2)t_k}{1-2t_k}\frac{g(x_k)}{g[w_k, x_k] + 2g[w_k, y_k]}, \tag{11}$$

   donde $w_k = x_k + \beta g(x_k)$ y $t_k = (g(y_k)/g(x_k))^{1/m}$.

4.2. Criterios técnicos de evaluación computacional

Para evaluar y contrastar cuantitativamente el desempeño de los métodos, se establecen los siguientes criterios:

1. **Criterio de Parada y Tolerancia Numérica:** Se adoptará la cota $|x_{k+1} - x_k| + |g(x_k)| < \text{TOL}$ con un tope de iteraciones $k_{\text{máx}} = 100$.
2. **Comportamiento en Flotante Standard (`float64`):** En `float64` el método se estanca cuando $w_k = x_k$, es decir, cuando $|g(x_k)| \lesssim \sqrt{\varepsilon_M |x_k|/(2\beta)} \approx 1.39 \times 10^{-7}$ para g_1. Se implementará una guarda condicional (`if w == x: break`) y se reportará el nivel de estancamiento observado como resultado empírico del experimento.
3. **Aritmética de Alta Precisión con `mpmath`:** Para reproducir las tolerancias del artículo ($\text{TOL} = 10^{-200}$) y alcanzar residuos extremos sin absorción numérica, se utilizará la biblioteca `mpmath` fijando la mantisa en 4000 dígitos decimales [10].
4. **Orden Computacional Empírico (ACOC):** Calcularemos el Orden de Convergencia Computacional Aproximado formulado por Cordero y Torregrosa [8]:

$$\text{ACOC} \approx \frac{\ln|(x_{k+1} - x_k)/(x_k - x_{k-1})|}{\ln|(x_k - x_{k-1})/(x_{k-1} - x_{k-2})|}. \tag{12}$$

   Un valor cercano a 4.0000 confirmará en la práctica la tasa de convergencia cuártica.
5. **Sensibilidad a la Semilla Inicial (x_0) y Estabilidad:** Se evaluará el radio de atracción probando semillas cercanas y lejanas (ej. $x_0 = 1.9$ y $x_0 = 3.0$ en g_1), verificando la presencia de bifurcaciones, oscilaciones o captura por raíces vecinas.
6. **Eficiencia Teórica y Tiempo de CPU:** Se contrastará el índice de Kung-Traub ($EI = p^{1/d}$), donde las familias propuestas ($EI = 4^{1/3} \approx 1.587$) superan a Newton modificado ($EI = 2^{1/2} \approx 1.414$), y se medirá el tiempo de procesador en milisegundos promediando 50 ejecuciones independientes.

4.3. Avance de implementación computacional preliminar en Python

Como evidencia de avance correspondiente a esta primera entrega, se implementaron las familias M_1 y M_2 en el script ejecutable `src/avance_fase1.py` del repositorio. La ejecución sobre la función $g_1(x)$ con semilla $x_0 = 1.9$ y 4000 dígitos de precisión en `mpmath` produjo los resultados sintetizados en el Cuadro 2:

Cuadro 2: Resultados preliminares de ejecución en Python (`src/avance_fase1.py`) para $g_1(x)$ con $x_0 = 1.9$.

Método	**Iteraciones (k)**	$\lvert x_{k+1} - x_k \rvert$	**Residuo** $\lvert g(x_{k+1}) \rvert$	**ACOC**	**Observación de Convergencia**
M_1 (Polinomial)	7	9.5190×10^{-537}	0.0	4.0000	Cumple tolerancia en $k = 7$; coincide con [1].
M_2 (Racional)	7	1.3790×10^{-776}	5.5295×10^{-2012}	4.0000	Cumple tolerancia en $k = 7$; paso y ACOC concuerdan con [1].

En el Cuadro 2, el paso se define como $|x_{k+1} - x_k|$ y el residuo como $|g(x_{k+1})|$, evaluado tras completar el paso corrector. El número de iteraciones ($k = 7$), la magnitud del paso y el ACOC (4.0000) concuerdan con los reportados en [1]; el residuo de M_2 difiere (5.53×10^{-2012} frente a 1.77×10^{-1796} del artículo), discrepancia posiblemente atribuible a particularidades en la definición del punto de evaluación del residuo en el entorno original de MATLAB frente a `mpmath`, comprometiéndonos a verificarlo en la Fase 2 imprimiendo el error absoluto $e_k = |x_k - \alpha|$ en cada iteración.

Declaración sobre el uso de herramientas de Inteligencia Artificial

En cumplimiento de los lineamientos de integridad académica de la Universidad Externado de Colombia y de la asignatura Métodos Numéricos, los autores declaramos que hemos utilizado herramientas de Inteligencia Artificial generativa como apoyo técnico en la diagramación del documento en LaTeX, la revisión gramatical de estilo, la estructuración preliminar del script exploratorio `src/avance_fase1.py` y la discusión analítica de conceptos teóricos y numéricos. Los integrantes del equipo comprendemos integralmente, sustentamos y asumimos la total autoría y responsabilidad del contenido matemático, la formulación computacional y los resultados de este documento.

Referencias

[1] Jerezano, J. H., Chicharro, F. I., & Garrido-Saez, N. (2026). Derivative-free optimal multiple-root finding family of iterative methods. *Computational and Applied Mathematics*, 45:304.

[2] Kung, H. T., & Traub, J. F. (1974). Optimal order of one-point and multipoint iteration. *Journal of the ACM*, 21(4), 643–651.

[3] Traub, J. F. (1964). *Iterative Methods for the Solution of Equations*. Prentice-Hall.

[4] Sharma, J. R., & Sharma, R. (2010). Modified Jarratt method for computing multiple roots. *Applied Mathematics and Computation*, 217, 878–881.

[5] Behl, R., Cordero, A., Motsa, S. S., & Torregrosa, J. R. (2016). An optimal fourth-order family of methods for multiple roots and its dynamics. *Numerical Algorithms*, 71, 775–796.

[6] Kumar, S., Kumar, D., Sharma, J. R., Cesarano, C., Agarwal, P., & Chu, Y. M. (2020). An optimal fourth order derivative-free numerical algorithm for multiple roots. *Symmetry*, 12, 1038.

[7] Behl, R., Bhalla, S., Magreñán, Á. A., & Moysi, A. (2021). An optimal derivative free family of Chebyshev–Halley's method for multiple zeros. *Mathematics*, 9, 546.

[8] Cordero, A., & Torregrosa, J. R. (2007). Variants of Newton's method using fifth-order quadrature formulas. *Applied Mathematics and Computation*, 190, 686–698.

[9] Turner, P. R., Arildsen, T., & Kavanagh, K. (2018). *Applied Scientific Computing with Python*. Springer-Verlag.

[10] Johansson, F. et al. (2024). *mpmath: Python library for arbitrary-precision floating-point arithmetic* (Version 1.4.1). `https://mpmath.org/`.